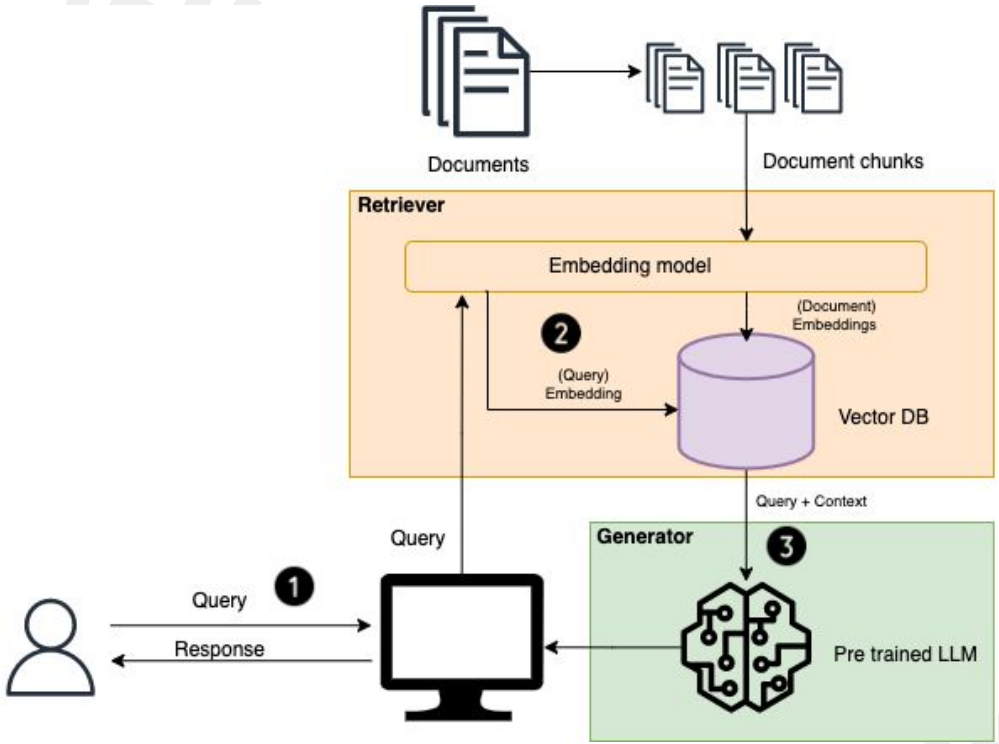
LLM

Parinya Duanklang

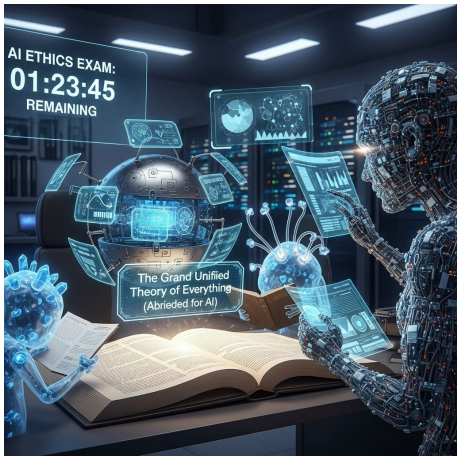
Retrieval-Augmented Generation (RAG)



https://www.lakera.ai/blog/retrieval-augmented-generation

The open-book test for Artificial Intelligence.

Retrieval-Augmented Generation (RAG) makes AI smarter by letting it **search** for real information from **trusted sources** before it answers you.



Without RAG:
AI only uses its **memory**. If it doesn't know the answer, it might **"hallucinate"** (make things up).

With RAG:
AI **checks the facts** from your provided files to give **accurate** and **up-to-date** answers.

Course Overview

Part 1: Foundation

- Introduction & Overview
- Tokenization & Embeddings
- Positional Encoding
- Query, Key, Value Matrices

Part 2: Attention Mechanism

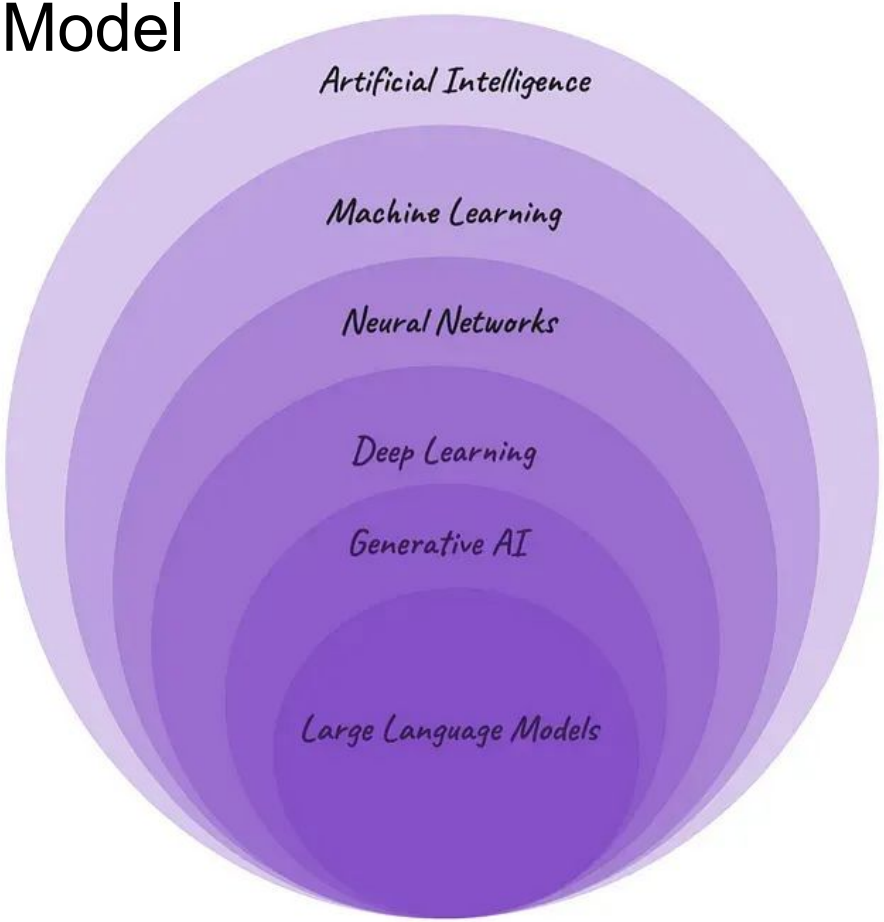
- Computing Attention Scores
- Softmax & Normalization
- Value vectors & Final Output

Part 3: Multi-Head Attention & Applications

- Multi-Head Attention
- Layer Stacking & Architecture
- Next Token Prediction Process
- Training & Applications

Defining the Large Language Model

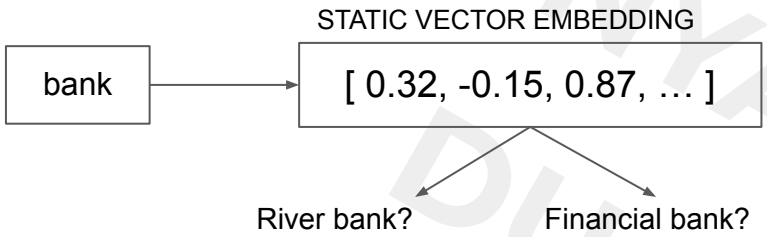
A Large Language Model (LLM) is a type of artificial intelligence (AI) designed to understand, process, and generate human-like text by analyzing massive datasets. They are built using deep learning techniques—specifically neural networks known as transformers—which allow them to learn language patterns, grammar, and context from vast amounts of data.



The Evolution of Natural Language Processing

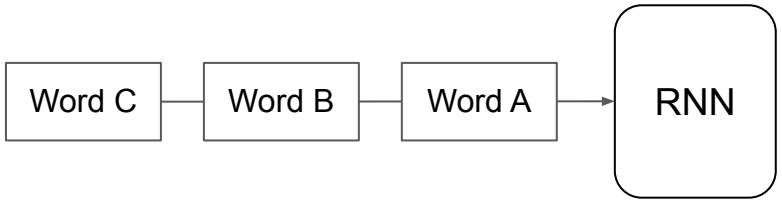
The Pre-Transformer Bottlenecks (2013-2017)

2013 Work2Vec (The Context Problem)



Limitation: "One size fits all" embeddings.
No contextual awareness.

2014 RNNs & LSTMs (The Speed Problem)



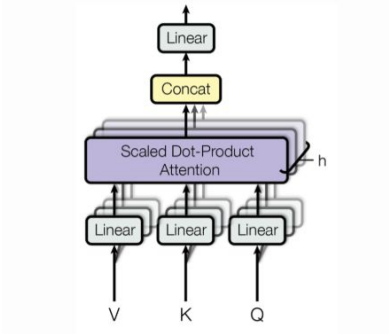
Mechanism: Sequential Information Processing

Critical Failure: Vanishing Gradient-the model "forgets" the beginning of long sentences

The Transformer Revolution (2017-Present)

2017: The Game Changer

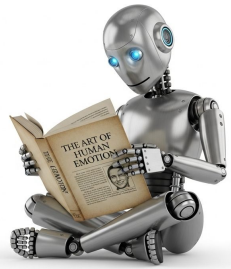
Attention Is All You Need



Innovation: Self-Attention Mechanism allows for massive scalability.

2018: Contextual Mastery

BERT & DistilBERT



Innovation: Bidirectional Understanding reads context from both directions.

2020+: Generative AI

GPT-3, ChatGPT, LLM

Gemini

Innovation: Emergent capabilities in reasoning and coding.

https://ar5iv.labs.arxiv.org/html/1706.03762

Deconstructing B.E.R.T.

Bidirectional

Analyzes context from **both directions** simultaneously. Example: In "The bank of the river," it uses **both** sides to define "bank".

Encoder

Focuses on **Understanding** rather than Generation. Builds rich numerical representations.

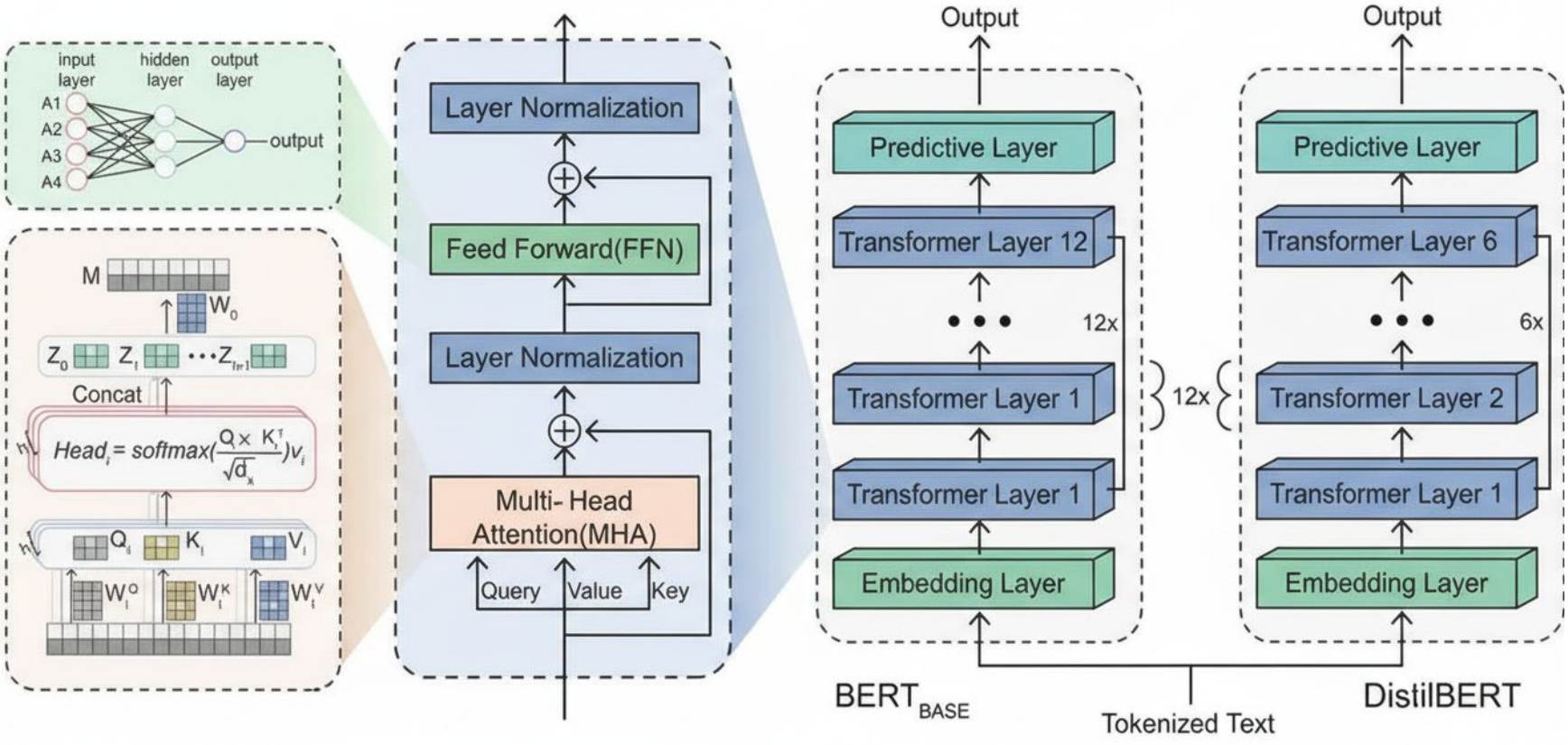
Representations

Converts words into high-dimensional vectors (768-dimensions). Captures deep semantic relationships.

Transformers

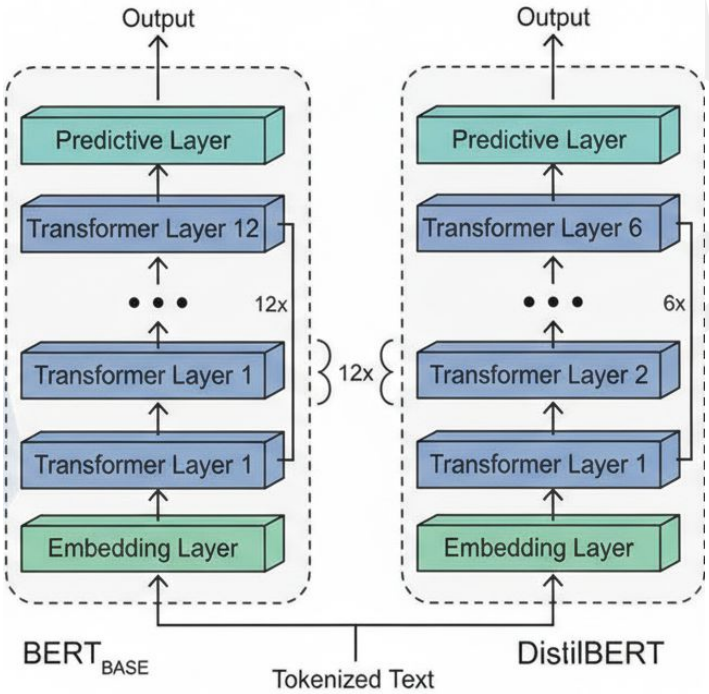
The underlying engine using **Attention Mechanisms**. Enables **parallel processing**.

BERT Architecture



https://www.researchgate.net/figure/Schematic-diagram-of-BERT-BASE-and-DistilBERT-model-architecture_fig1_382939584

Efficiency vs. Power: BERT vs. DistilBERT

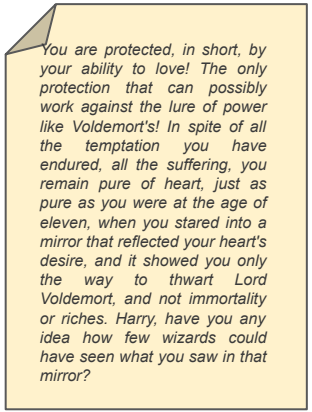


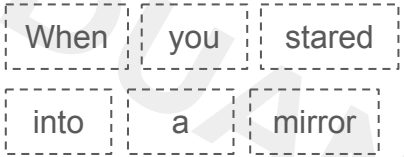
Feature	BERT Base	DistilBERT
Layers	12	6 (-50%)
Attention Heads	144 (12x12)	72 (6x12)
Parameters	110M	66M (-40%)
Model Size	440 MB	255 MB (-42%)
Speed	1x (Baseline)	1.6x (60% faster)
Performance	100%	97%

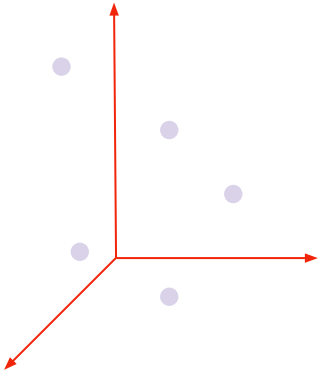
Tokenization & Embeddings

Tokenization & Embeddings

Transforming unstructured text into numerical vectors.







Raw Input

When you stared into a …

Tokenization

Breaking down sentences

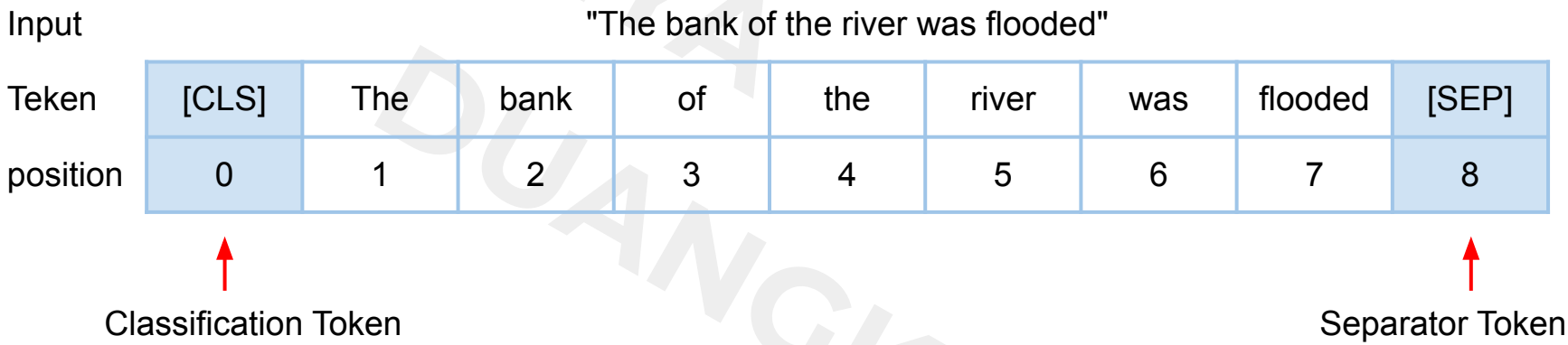
Indexing

Assigning integer IDs

Embedding

Vector Lookup

Tokenization: The Atomic Units of Language



WordPiece Tokenization: Efficiency & Flexibility

Traditional Word Split

“unhappiness” → [UNKNOWN TOKEN]

Subword Tokenization

“unhappiness” → [“un”, “##happiness”]

"playing" → ["play", "##ing"]

"unbelievable" → ["un", "##believ", "##able"]

"COVID-19" → ["COVID", "-", "19"]

- Smaller vocabulary
- Handle rare words
- Share knowledge (un- + happy)

Vocabulary & Token IDs

Every unique token maps to a permanent integer ID.

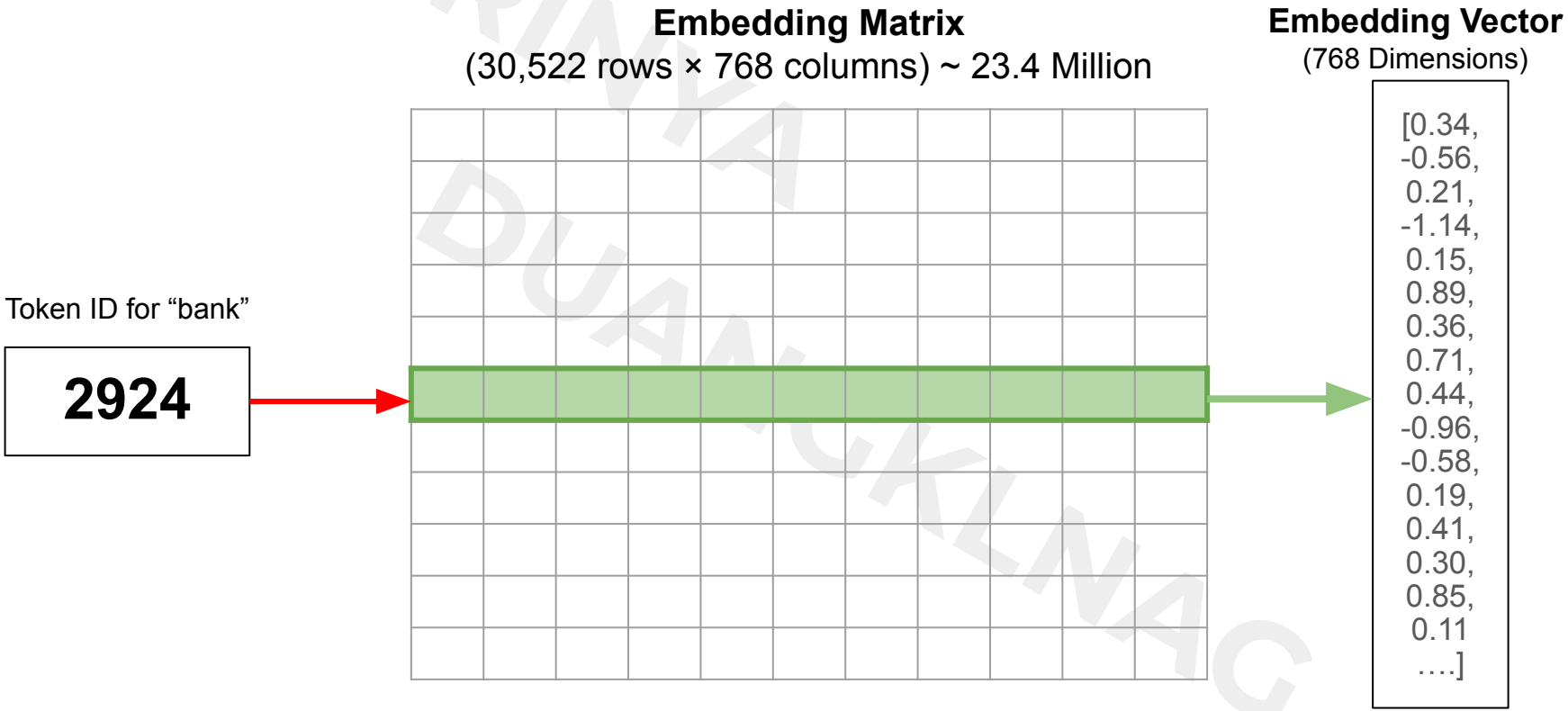
Input "The bank of the river was flooded"

Teken	[CLS]	The	bank	of	the	river	was	flooded	[SEP]
Token ID	101	1996	2924	1997	1996	2314	2001	13042	102
Segment IDs	0	0	0	0	0	0	0	0	0

BERT Vocabulary Facts

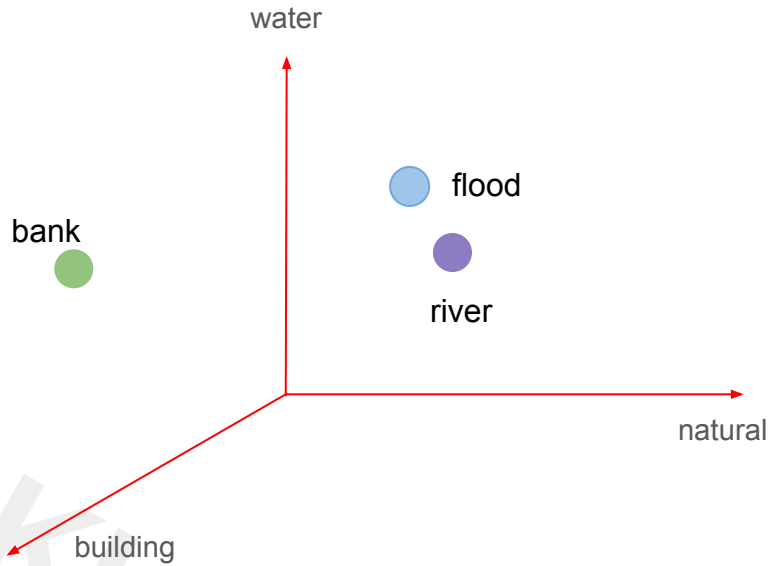
- Dictionary Size: 30,522 unique tokens
- Mapping Logic: Strict 1-to-1 static mapping

Word Embeddings: The Lookup Layer



Embeddings Capture Meaning: The Geometry of Language

Token	Token ID	**Embedding Vector** (768D)
[CLS]	101	[1.0, 0.2, 0.1, …]
The	1996	[0.2, 0.8, 0.1, …]
bank	2924	[0.2, 0.1, 0.9, …]
of	1997	[0.9, 1.0, 0.2, …]
the	1996	[0.2, 0.8, 0.1, …]
river	2314	[0.0, 0.9, 0.9, …]
was	2001	[0.3, 0.8, 0.9, …]
flooded	13042	[0.1, 0.9, 0.8, …]
[SEP]	102	[0.0, 0.3, 0.1, …]



Euclidean Distance $d(p,q) = \sqrt{\sum_{i=1}^{n}(q_i - p_i)^2}$

Positional Encoding

The Problem of Order Invariance

The dog chased the cat ≠ **The cat chased the dog**

Same words. Same embeddings. Different meanings.

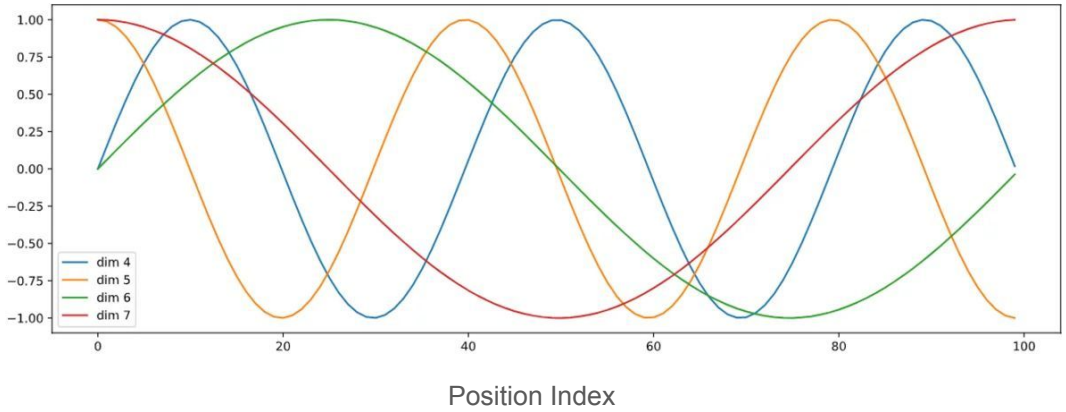
Without position data, the model sees these as identical "bags of words".

Positional Encoding: Adding Location Data

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

Where: pos = position (0, 1, 2, ...)
i = dimension index
d = embedding dimension (768)

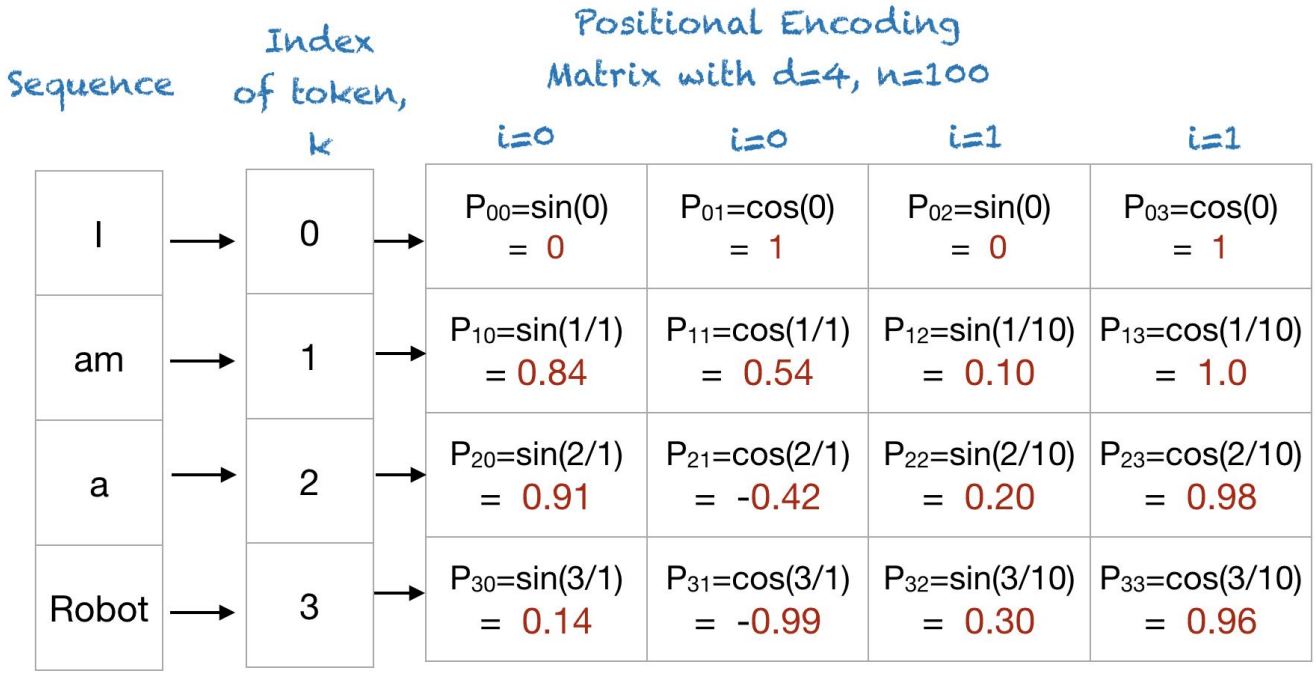


1. **Unique pattern per position**
   pos=0 ≠ pos=1 ≠ pos=2

2. **Smooth changes**
   Nearby positions → similar vectors
   pos=5 similar to pos=6

3. **Bounded values**
   Always between -1 and +1
   Stable for training

4. **Extrapolation**
   Can handle longer sequences
   Not seen in training



Positional Encoding Matrix for the sequence 'I am a robot'

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d}}\right)$$

$$PE(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d}}\right)$$

https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/

The Final Input Representation

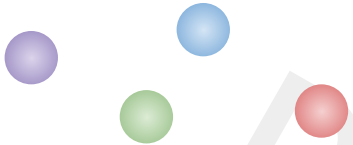
The Transformer now receives this combined representation:

Token	Word Embedding		Segment embedding		Positional Encoding		Final Input
[CLS]	[1.0, 0.2, 0.1, …]	+	[0.55, -0.21, 0.04, ...]	+	[0.0, 1.0, 0.0, …]	=	[1.0, 1.2, 0.1, …]
The	[0.2, 0.8, 0.1, …]	+	[0.55, -0.21, 0.04, ...]	+	[0.84, 0.54, 0.91, …]	=	[1.04 1.34 1.01, ...]
bank	[0.2, 0.1, 0.9, …]	+	[0.55, -0.21, 0.04, ...]	+	[0.91, -0.42, 0.99, …]	=	[1.11, -0.32, 1.89, …]
of	[0.9, 1.0, 0.2, …]	+	[0.55, -0.21, 0.04, ...]	+	[0.14, -0.99, 0.96, …]	=	[1.04, 0.01, 1.16, …]
the	[0.2, 0.8, 0.1, …]	+	[0.55, -0.21, 0.04, ...]	+	[-0.96, 0.28, 0.65, …]	=	[-0.96, 1.18, 1.55, …]
…	…		…		…		…
[SEP]	[1.0, 0.5, 0.6, …]	+	[0.55, -0.21, 0.04, ...]	+	[0.25, 0.38, 0.69, …]	=	[1.25, 0.88, 1.29, …]

Query, Key, Value Matrices

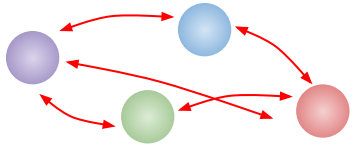
The Attention Engine (Q, K, V)

The Problem: Static Embedding



Embedding provide “What + Where” but lack “Relationships”. A static vector for “bank” is identical in every context.

The Solution



Goal: Transform static inputs into “Attention-Ready” vectors that can “talk” to each other.

The "Three Matrices" Concept: Why we need three different views of the same word.

- **Query** (W_Q) **:** *"What am I looking for?"* (The searcher).
- **Key** (W_K) **:** *"What do I have to offer?"* (The label).
- **Value** (W_V) **:** *"What is my actual content?"* (The information).

The Central Question: What is Attention Solving?

When processing the word “bank”, the model must answer three critical question:

The **bank** of the **river** was **flooded**

1. Which words are relevant?

Search Target “river”, “flooded”.

2. To what degree?

Assign Importance Scores (Weight).
River: 85%, of: 5%.

3. What info to extract?

Extract geographical context, ignore financial context.

The Database Analogy: A Mental for Q, K, V

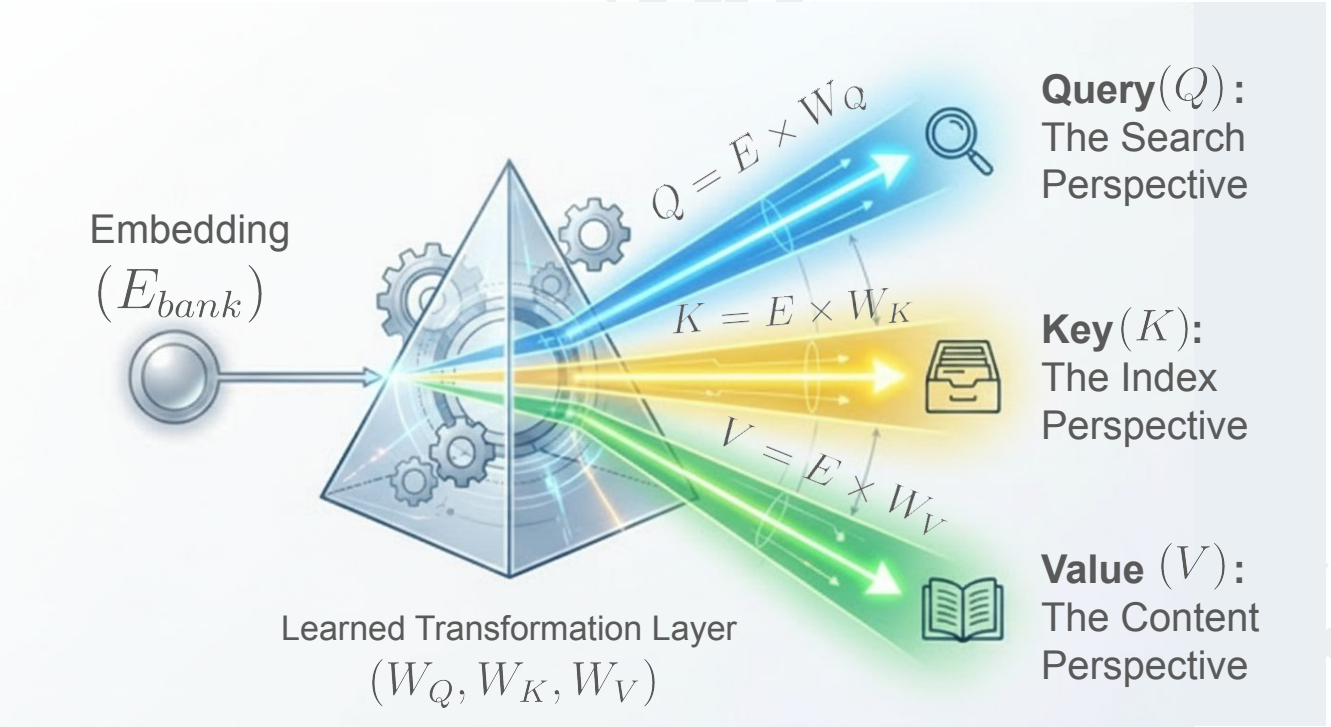
Query (Q) **:**
The Search Term

Key (K) **:**
The Catalog Title

Value (V) **:**
The Book Content

Conceptual Flow: The Query term matches the relevant Key, which then unlocks and retrieves the actual Value (content) associated with that key.

The Power of Perspective: Why Three Matrices?



The Limitation: Static embeddings are rigid. "Bank" always equals "Bank".

The Solution: Learned matrices (W_Q, W_K, W_V) create dynamic views.

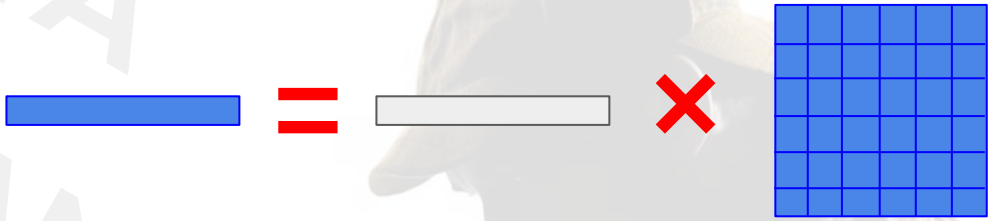
The Benefit: One word can now play three roles simultaneously.

Deep Drive: The Query Matrix (W_Q) - "What am I looking for?"

Role: The Searcher. Defines criteria for what the token needs.

Example:
Q_{bank} : "Looking for geographical indicators."
Q_{river} : "Looking for flow/water-related adjectives."
Q_{was} : "Looking for the subject of the sentence."

$$Q = E \times W_Q$$



Query vector
[1 × 64]

Embedding
[1 × 768]

Query Matrix
[768 × 64]

Compresses 768 dimensions into a focused 64-dimensional **search vector**.

Deep Drive: The Key Matrix (W_K)- "What do I have to offer?"

Role: The Label. Acts as a descriptor for others to find.

Example:
K_{bank} **:** "I contain geographical features."
K_{river}**:** "I contain location properties."
K_{was} **:** "I am just a grammatical connector."

$$K = E \times W_K$$

Key vector
[1 × 64]

Embedding
[1 × 768]

Key Matrix
[768 × 64]

Same shape as Query (768 x 64) but learns different weights to **facilitate matching.**

Deep Drive: The Value Matrix (W_V) - "What is my actual content ?"

$$V = E \times W_V$$

Role: The Content. The meaningful information passed forward.

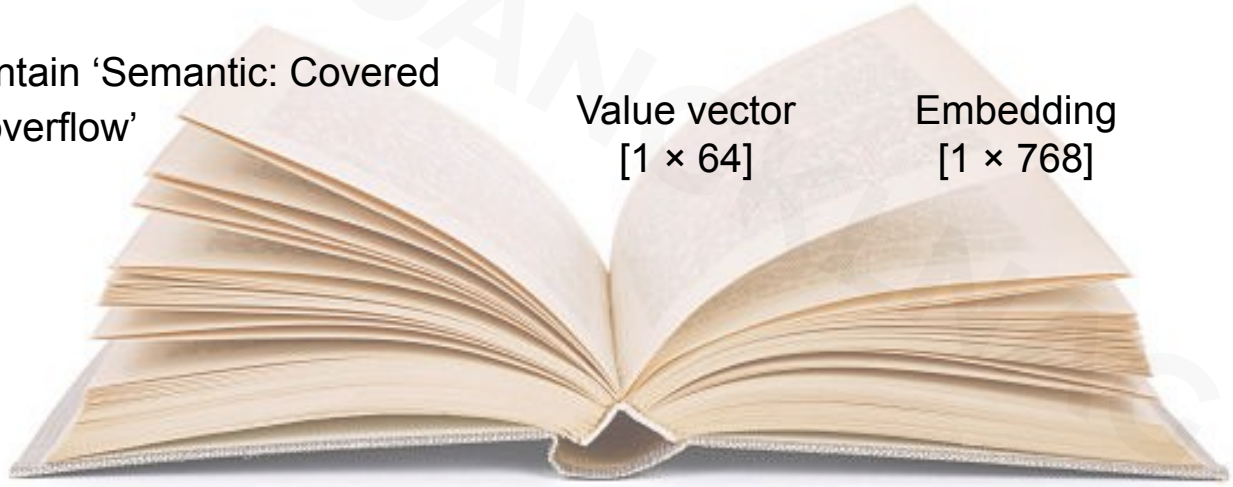
V_{river} contain 'Semantic: Body of water, natural feature'.

$V_{flooded}$ Contain 'Semantic: Covered with water, overflow'

Value vector
[1 × 64]

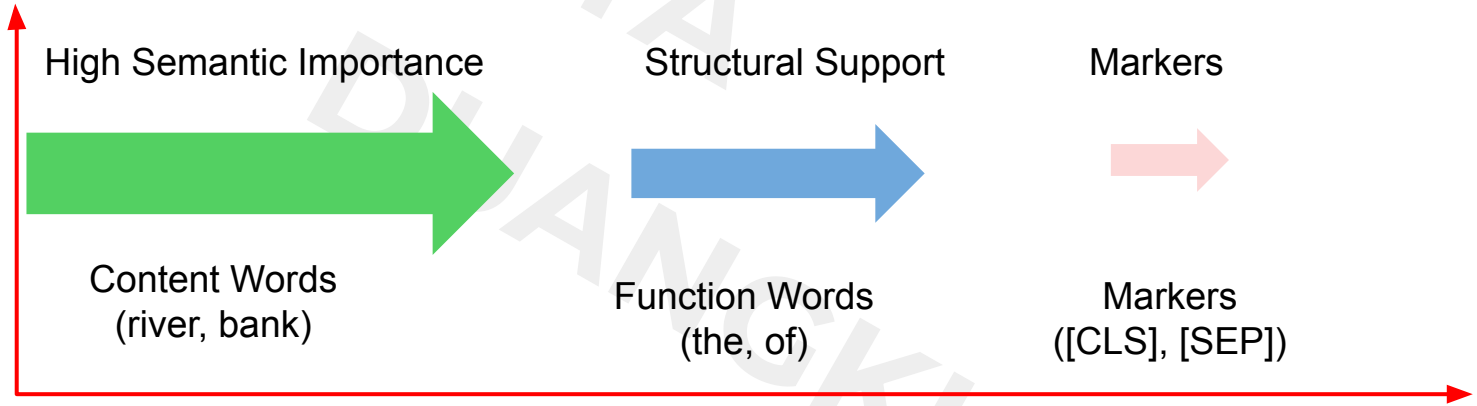
Embedding
[1 × 768]

Value Matrix
[768 × 64]



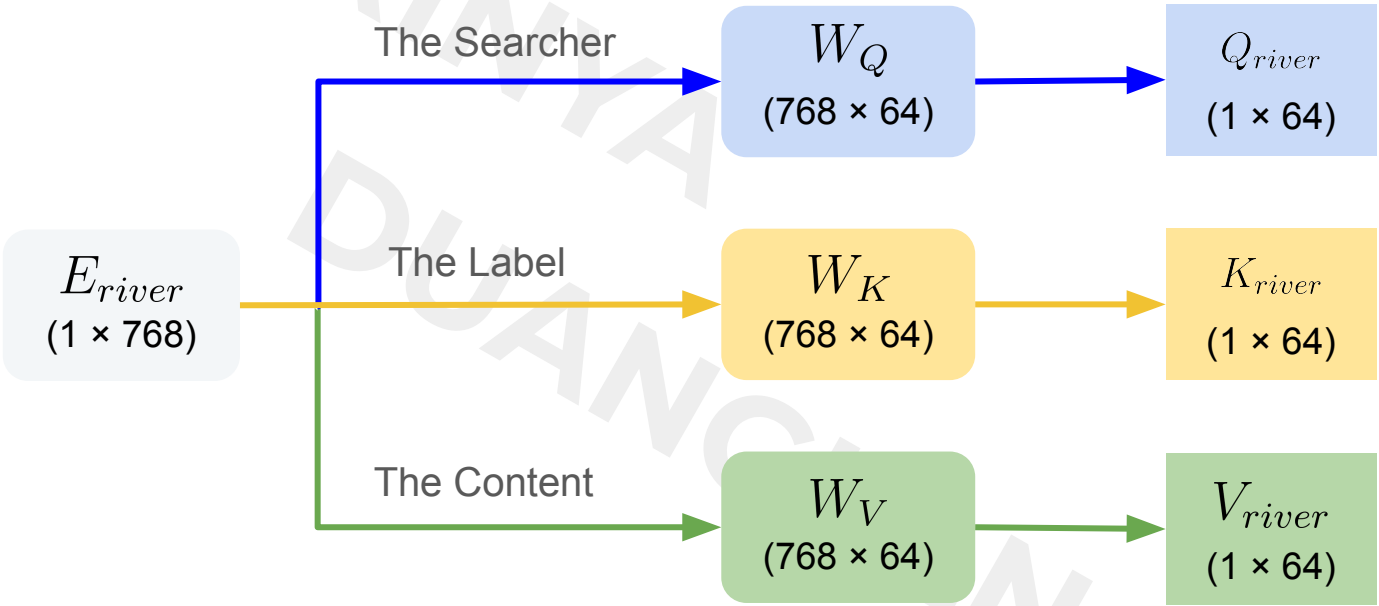
Computing Value Vectors: The Semantic Payload

$$V = E \times W_V$$



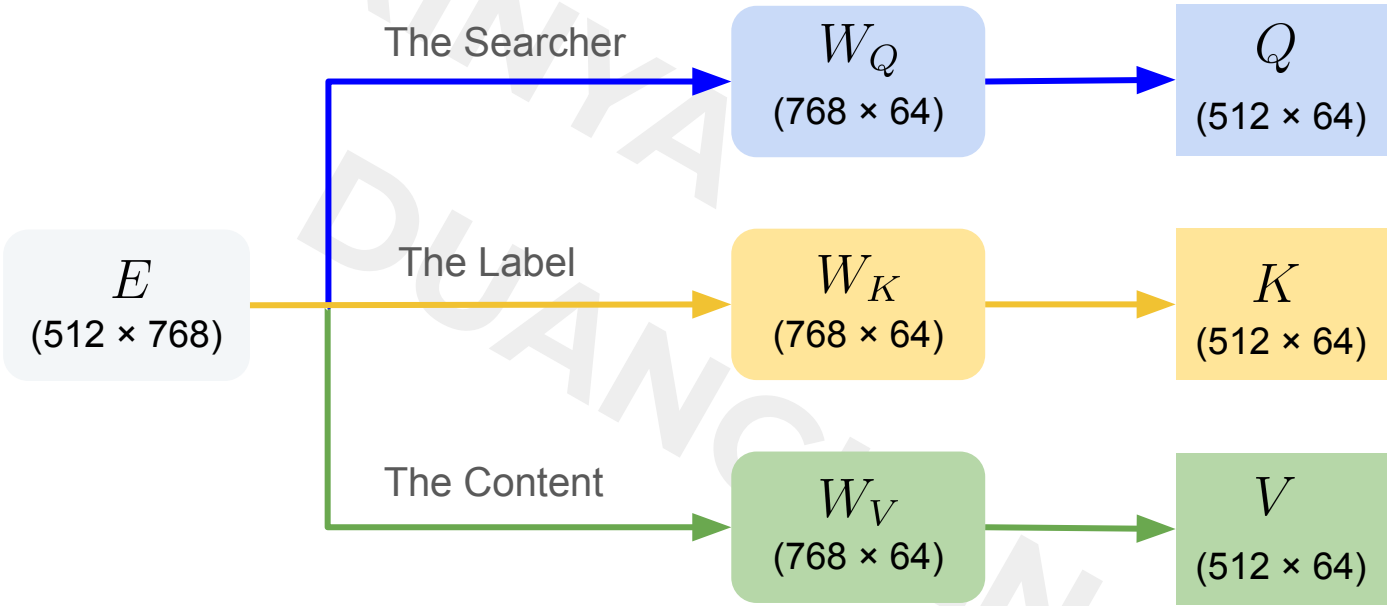
Insight: The model learns to store heavy semantic meaning in content words, while function words serve as structural glue.

Visualizing the Flow: One Input, Three Destinies



Computation Cost (1 token) = 49,152 ops.
Total (3 Matics × 49,152) = 147,456 ops per token

Scaling Up: Processing the Full Sequence



Scale: 512 tokens.
Operations: 512 tokens × 147,456 ops per token = **75,497,472 ops.**
This happens in parallel for every single token.

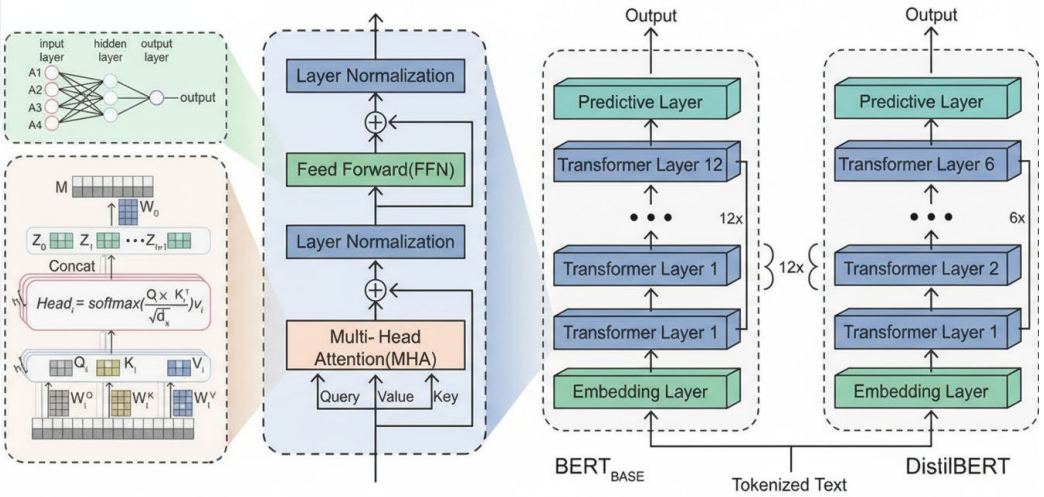
The Engineering Reality: BERT & DistilBERT

Architecture: 12 Independent Attention Heads per layer.

Matrix Count: 36 matrices per layer (12 heads × 3 metrice).

just for 1 attention layer

- **Parameters:** ≈ 1.7 Million parameters
- **Operations**: ≈ 900 Million ops.



Why we need GPUs: Massive Parallelization

Hand on (Day 1: Morning)

Tech Stack

https://colab.google

https://www.langchain.com

https://groq.com

https://ollama.com

https://huggingface.co

https://openai.com

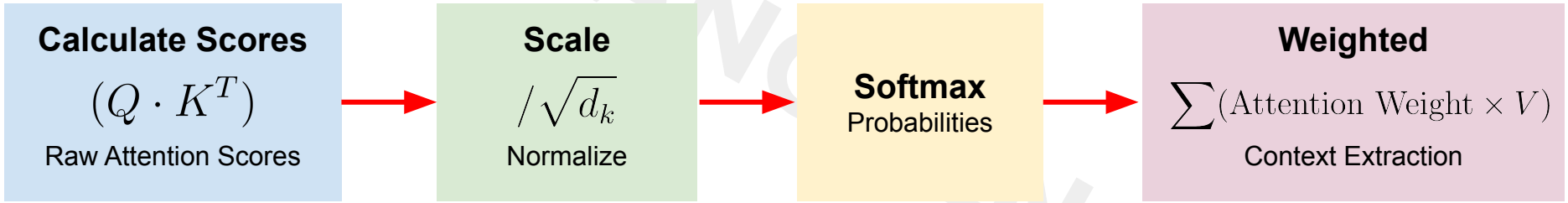
LangChain Basics

- Understand the difference between standard Prompts and Prompt Templates.
- Learn how to define AI Personas (expert systems) using System Messages.
- Learn how to manage Conversation History so the AI can remember previous interactions.
- Practice using LCEL (LangChain Expression Language) to connect different components together.
- Build Workflows using Sequential (processing in order) and Parallel (processing simultaneously) patterns.

Computing Attention Scores

The Attention Formula: A Roadmap

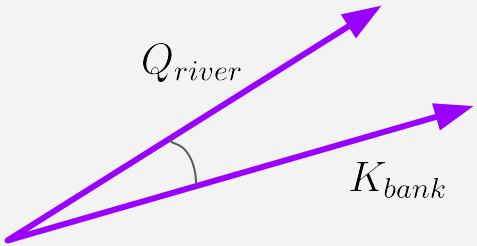
$$Attention(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) V$$



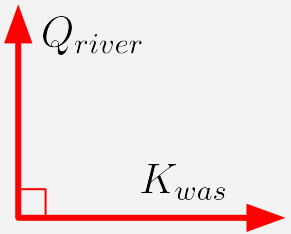
Computing Attention Scores

How the model determines **relationships** between words.

The Mechanism: Dot Product $(Q \cdot K^T)$



Strong Attention



Low Attention

The **Dot Product** measures alignment. Vectors pointing in the same direction yield a high score.

The “River” Example: Raw Attention Scores

Query: ‘river’ vs. Sentence Key

Rank	Token	$Q_{river} \cdot K_{token}^T$	Interpretation
1	river	0.216	Self-attention (highest)
2	bank	0.204	Geographical relation
3	flooded	0.096	Water-related event
4	was	0.000	Neutral
5	[SEP]	-0.012	Irrelevant (Low)
6	The	-0.258	Irrelevant (Low)
6	the	-0.258	Irrelevant (Low)
8	[CLS]	-0.774	Irrelevant (Low)
9	of	-0.798	Irrelevant (Low)

The model correctly identifies "bank" and "flooded" as highly relevant, while ignoring grammatical words like "of" and "the".

The Massive Scale of Attention

Queries: ALL tokens (Q_CLS, Q_The, ..., Q_SEP)
Keys: ALL tokens
9 queries × 9 keys = 81 scores!

Complete attention matrix:

	[CSL]	The	bank	…	[SEP]
[CSL]	S_{00}	S_{01}	S_{02}	…	S_{08}
The	S_{10}	S_{11}	S_{12}	…	S_{18}
bank	S_{20}	S_{21}	S_{22}	…	S_{28}
…	…	…	…	…	…
[SEP]	S_{80}	S_{81}	S_{82}	…	S_{88}

512 × 512 = 262,144 scores!
Per attention head!

- × 12 heads = 3,145,728 scores
- × 12 layers = 37,748,736 total!

Billions of attention computations!
Why GPUs are essential!

Total: ~37.7 Million attention scores calculated per pass

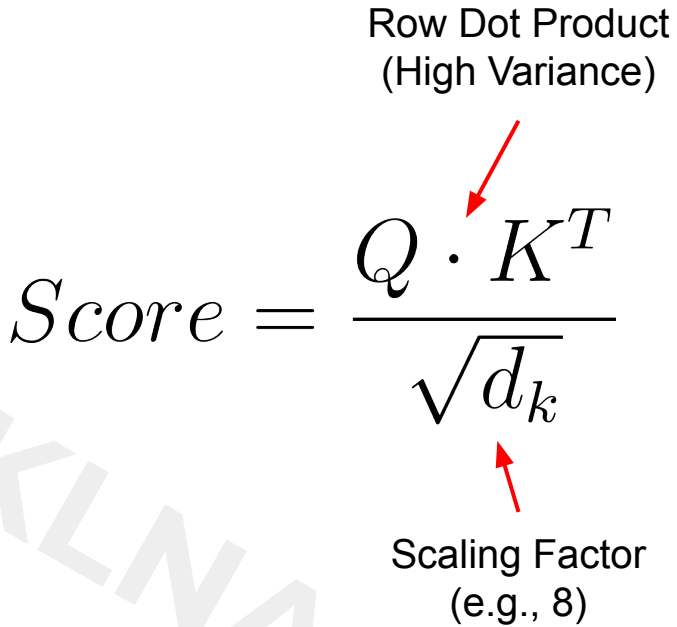
Normalization & Softmax

Normalization: Stabilizing the signal

The Role of Normalization

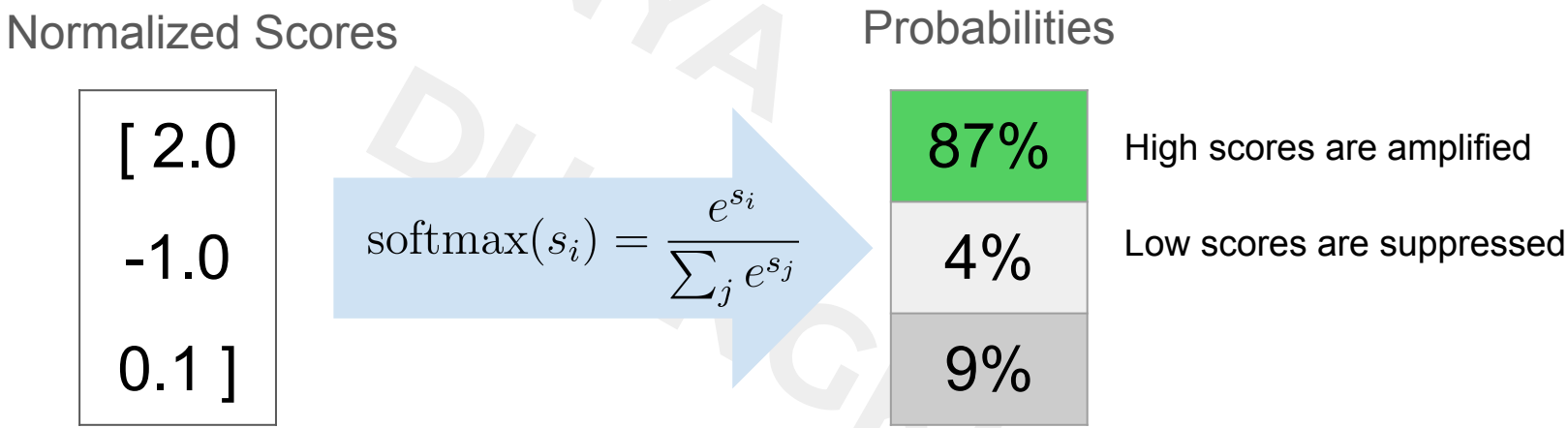
Raw attention scores are volatile. In high-dimensional spaces (like d=64), dot products can explode at products can explode in magnitude, destabilizing the training process.

Before interpretation, we must variance-scale the signal.



The Softmax Function: Decision Time

Convert the scaled scores into probabilities



Now the model has a clear **Attention Map** telling it exactly what percentage of focus to give each word.

Final Attention Weights: The Focus Distribution

How the token "river" allocates its attention

Token	Raw Attention Scores	Attention Weight
river	0.216	0.1365
bank	0.204	0.1355
flooded	0.096	0.1273
was	0.000	0.1205
[SEP]	-0.012	0.1196
The	-0.258	0.1038
the	-0.258	0.1038
[CLS]	-0.774	0.0770
of	-0.798	0.0759

Top Signal (~40%) High Semantic Relevance

Suppressed Signal Grammar & Noise

Value Vectors & Final Output

The Weighted Aggregation: The Result for "river"

Goal: Gather semantic information from the entire sentence to create a new, context-aware vector.

$$Output_{river} = \sum(\text{Attention Weight} \times V)$$

Token	Attention Weight	V_{token}	Weighted Vector
river	0.1365	× [2.07, 2.25,..., 2.43]	[0.2826, 0.3071,..., 0.3317]
bank	0.1355	× [1.41, 1.53,..., 1.65]	[0.1911, 0.2073,..., 0.2236]
flooded	0.1273	× [2.01, 2.19,..., 2.37]	[0.2559, 0.2788,..., 0.3017]
was	0.1205	× [2.18, 2.38,..., 2.58]	[0.2627, 0.2868,..., 0.3109]
[SEP]	0.1196	× [0.43, 0.47,..., 0.51]	[0.0514, 0.0562,..., 0.0610]
The	0.1038	× [1.07, 1.18,..., 1.29]	[0.1111, 0.1225,..., 0.1339]
the	0.1038	× [1.07, 1.18,..., 1.29]	[0.1111, 0.1225,..., 0.1339]
[CLS]	0.0770	× [1.03, 1.16,..., 1.29]	[0.0793, 0.0893,..., 0.0993]
of	0.0759	× [1.89, 2.10,..., 2.31]	[0.1434, 0.1594,..., 0.1753]

$Output_{river}$ = [1.4886, 1.6299, …, 1.7713]

Context for Every Token

After one Attention Layer, the model produces a complete matrix of updated vectors:

Output	context-aware concept
[CLS]	Now understands the overall sentiment of the sentence.
bank	Now "knows" it is a riverbank because it looked at *river*.
river	(The one we calculated!) Now enriched with *bank* and *flooded*.
was/flooded	Now carry the subject/object relationship.
the	They act as **Grammatical Glue**.
[SEP]	the entire sentence boundary.

Every word enters the layer as a **dictionary definition** and leaves as a **context-aware concept**.

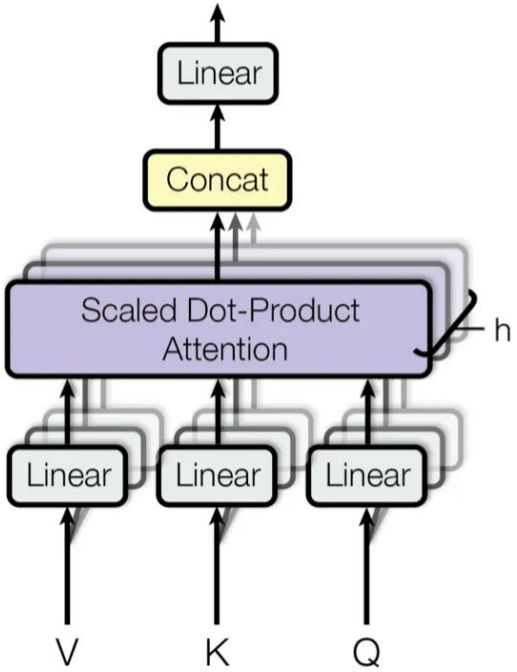
Multi-Head Attention

The Problem with a Single Perspective

The Limitation (Single-Head)

A Single Attention Head creates a valid "context-aware" representation.

Limitation: It has tunnel vision. It focuses on only one type of relationship at a time (e.g., Grammar OR Meaning).



The Solution (Multi-Head)

- 12 Attention Heads working in parallel.
- 12 different sets of Query, Key, Value matrices.
- Result: 12 unique attention patterns captured simultaneously.

https://aiml.com/what-is-multi-head-attention-and-how-does-it-improve-model-performance-over-single-attention-head/

One Word, Multiple Meanings

Head 1: Syntax

Focus: Structure.
"bank" is part of a prepositional phrase.

Head 2: Semantics

Focus: Context.
Detects water-related geography.

The bank of the river was flooded.

Head 3: Identity

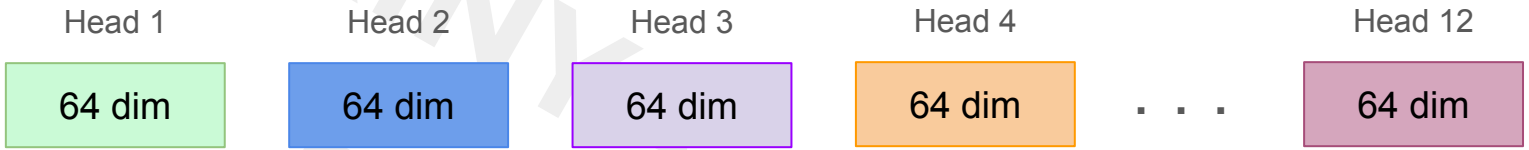
Focus: Self.
Preserves original word identity.

Head 4: Logic

Focus: Relationship.
Connects subject to verb.

The Concatenation Process

Step 1. Individual Head Outputs



Each of the 12 Attention Heads completes its own "Weighted Sum" calculation for the token **"river"**.

Snap Together

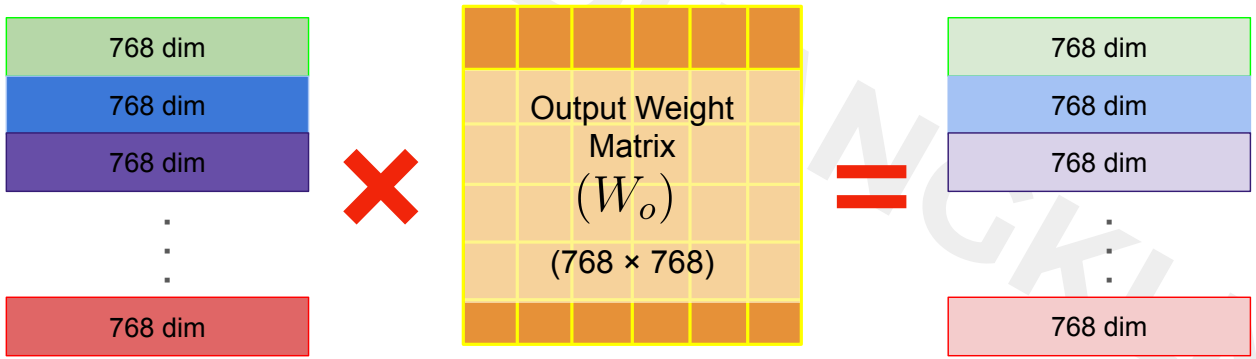
Step 2. Concatenation: The Big Merge

Result: We return to the original BERT embedding size (**768D**), but now every dimension is packed with deep contextual knowledge.

The Final Touch: Output Projection (W_o)

$$MultiHead(Q, K, V) = \text{Concat}(head_1, \ldots, head_{12}) \times W_O$$

Concatenated Output
from multi-Heads



Final Multi-Head Output
Integrated & Refined Context.

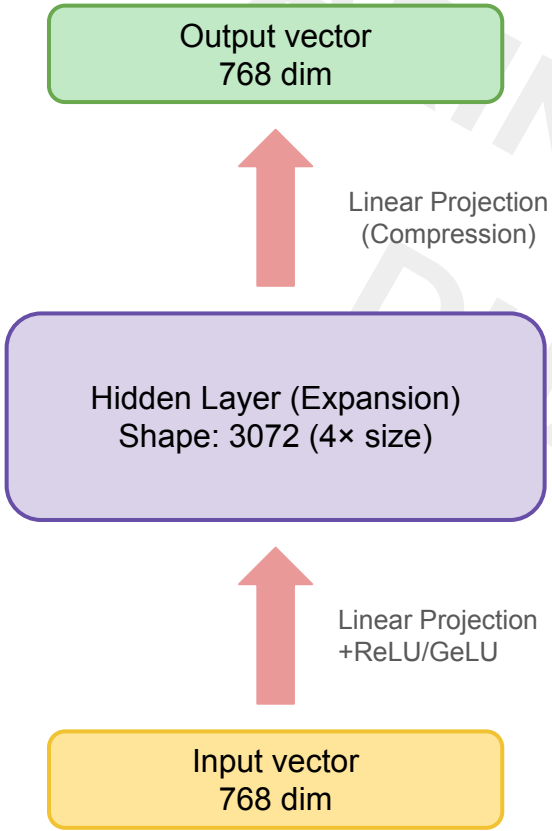
The Editor
Learned parameters that decide which head's info is most important.

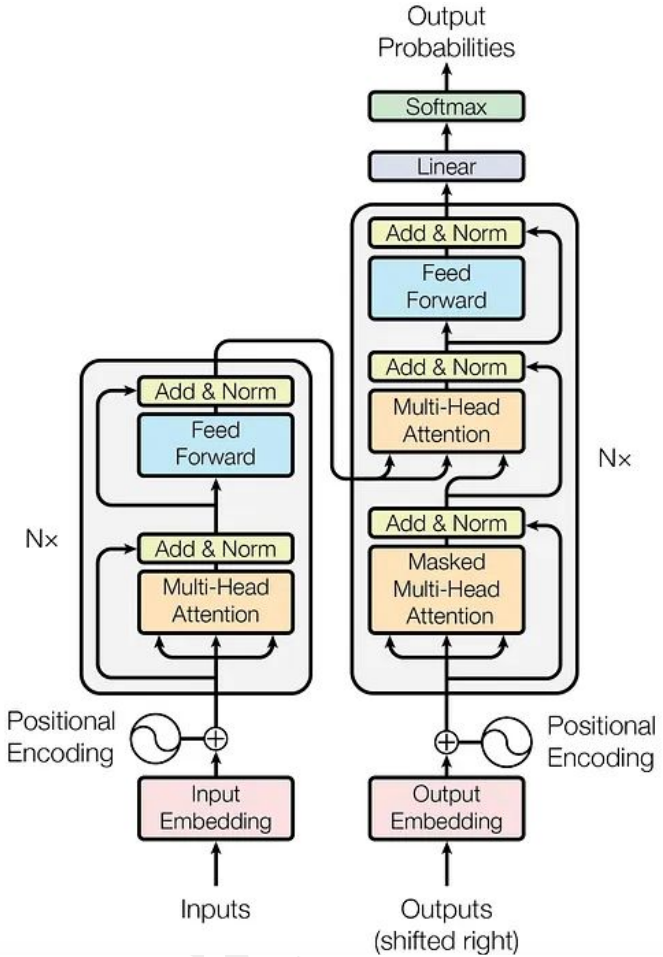
Why do we need W_o?

1. Inter-Head Communication: Allows the Grammar head to 'talk' to the Semantic head.
2. Refinement: Filters and mixes insights for the next layer.

Layer Stacking & Architecture

The Feed-Forward Network (FFN)



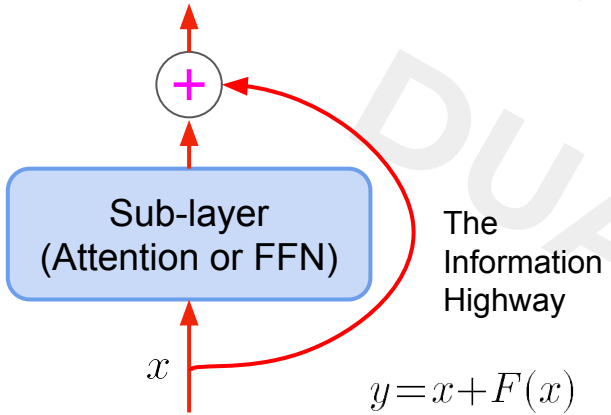


https://quantpedia.com/bert-model-bidirectional-encoder-representations-from-transformers/

Add & Norm: Stabilizing the Deep Network

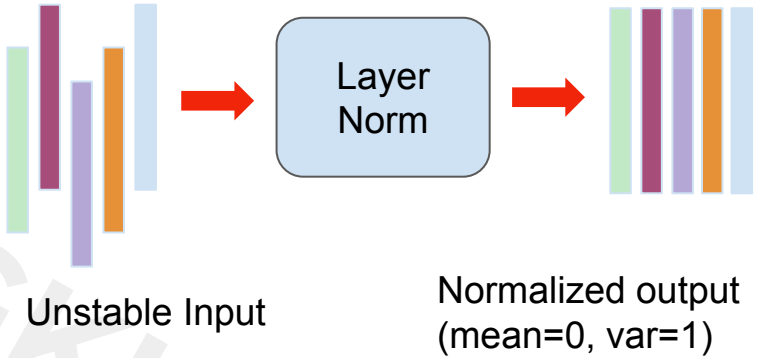
Guardrails against vanishing gradients and training instability.

Component 1: Residual Connections ('Add')



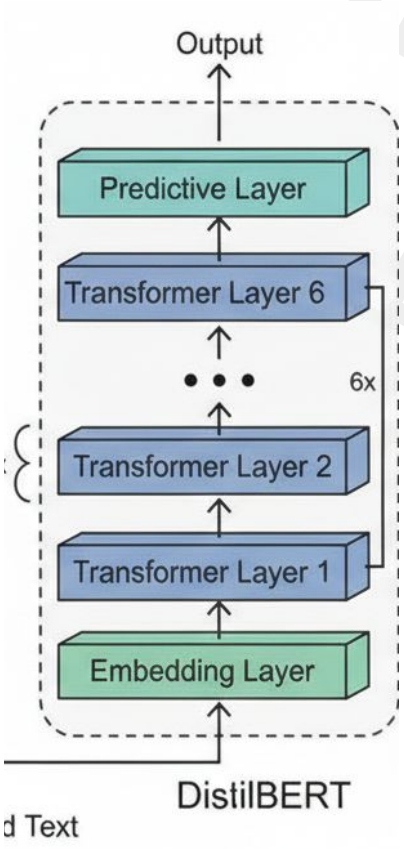
The Mechanism: $x + Sublayer(x)$ Allows gradients to flow through the network without vanishing, preserving the original signal.

Component 2: Layer Normalization ('Norm')



The Benefit: Standardizes inputs to ensure stable training dynamics, allowing the model to learn faster and deeper.

Stacking the Layers



Layer 5-6: Global meaning & Task readiness.
Knowledge: The entire sentence describes a natural event; ready for classification.

Layer 4: Actions & Roles.
Knowledge: The "bank" is the entity being affected by "flooding."

Layer 3: Disambiguation.
Knowledge: "bank" means geographical edge (not a building).

Layer 2: Phrases & Adjacent words
Knowledge: "bank" is a singular noun.

Layer 1: Grammar & Parts of speech.
Knowledge: "bank" is a singular noun.

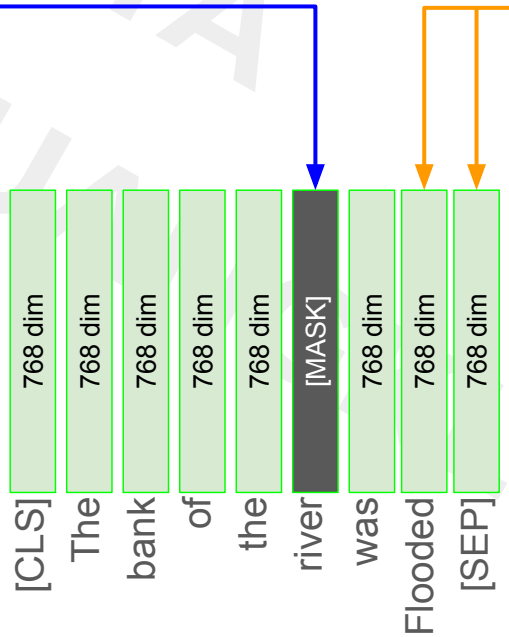
Next Token Prediction Process

Extracting Information

Choosing the specific 768D vector (selection token) that holds the answer for your task

Masked Token Prediction (BERT-style)

- **Goal:** Fill in a missing word within a sentence.
- **Selection:** Pick the representation at the [MASK] position.
- **Logic:** Context gathered from both left and right.



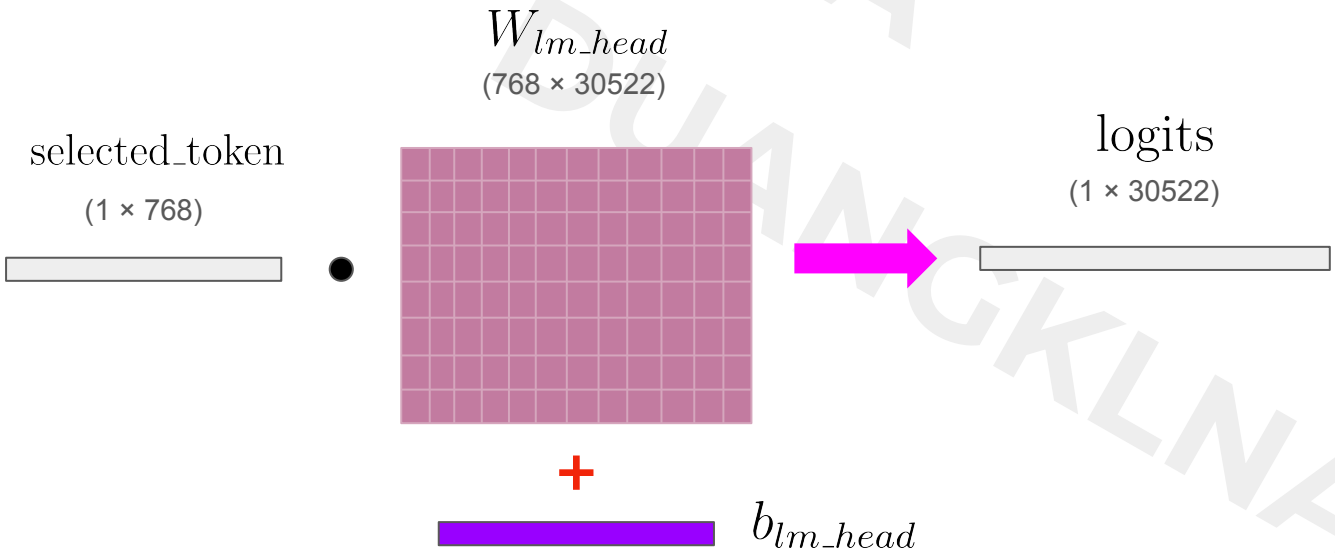
Next Token Prediction (GPT-style)

- **Goal:** Predict the very next word in a sequence.
- **Selection:** Pick the **Last Token** representation.
- **Logic:** The last token has "seen" everything before it

Linear Projection: The Language Model Head

Concept: Transforming the 768D hidden representation into raw scores for the entire vocabulary.

$$\text{logits} = \text{selected\_token} \cdot W_{lm\_head} + b_{lm\_head}$$



Understanding Logits

- Higher Score: More likely to be the correct word.
- Negative Score: Very unlikely to fit the context.

Softmax Activation

Converting raw logits into a probability distribution.

$$P(\text{token}_i) = \frac{e^{\text{logit}_i}}{\sum_j e^{\text{logit}_j}}$$

Word	Logits	Probabilities
the	2.1	0.02
area	5.8	0.11
severely	7.2	0.35
quickly	5.1	0.08
by	1.8	0.01
…	…	…

Decoding Strategy: How AI Decides its Next Word

Greedy Decoding

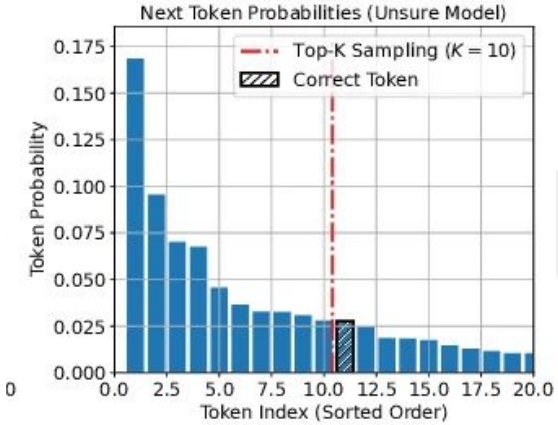
- **Mechanism:** Always picks the word with the **highest probability**
- **Best for:** Factual QA, Classification.

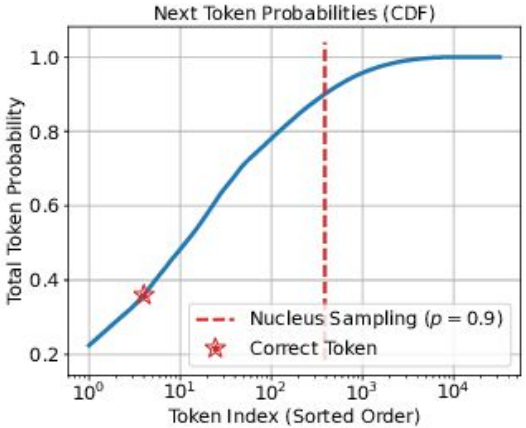
Top-k Sampling

- **Mechanism:** Filters the **top k words**, renormalizes their probabilities, and samples from them.
- **Best for:** Balancing quality and diversity.

Top-p (Nucleus) Sampling

- **Mechanism:** Picks the smallest set of words whose **cumulative probability exceeds p** (e.g., $p=0.9$).
- **Best for:** Professional chatbots





https://www.researchgate.net/figure/Top-K-sampling-is-insensitive-to-model-confidence_fig2_370841754

https://www.researchgate.net/figure/Nucleus-sampling-can-have-unpredictable-behavior-and-lead-to-many-low-probability-tokens_fig3_370841754

Training & Applications

The Pre-training Process: Building the 'Brain'

Learning general language patterns from 3.3 Billion words without human labels.

Wikipedia

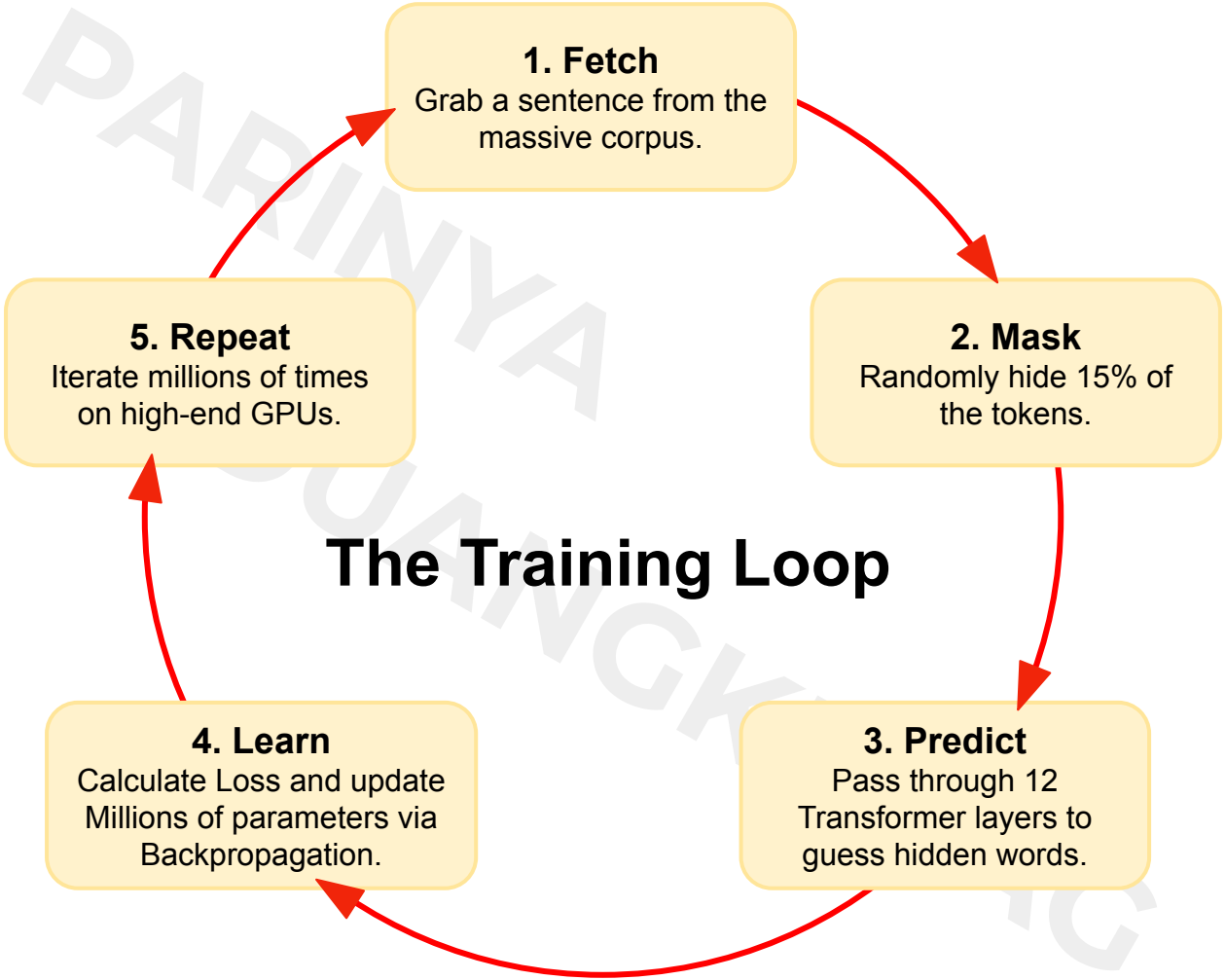
2.5 Billion words (Facts, structure, formal tone).

BookCorpus

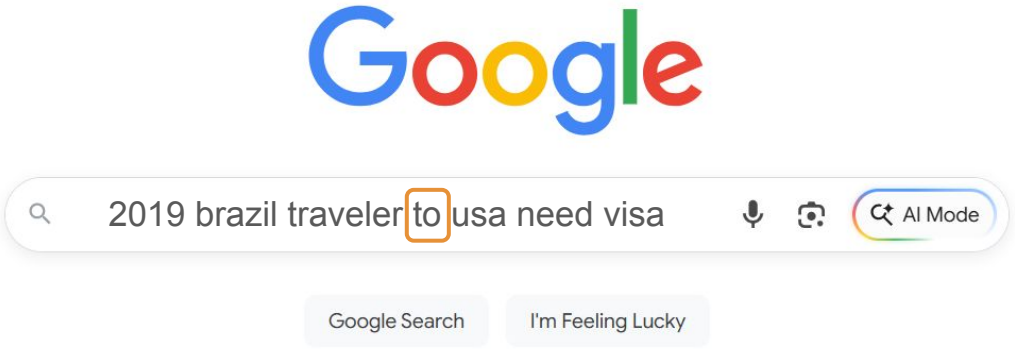
800 Million words (Stories, emotions, complex narratives).

The Outcome: A 'Generalist' model that understands grammar, context, and world knowledge, ready for hire.

https://en.wikipedia.org/wiki/Wikipedia
https://huggingface.co/datasets/SamuelYang/bookcorpus/tree/main


1. Fetch
Grab a sentence from the massive corpus.
2. Mask
Randomly hide 15% of the tokens.
3. Predict
Pass through 12 Transformer layers to guess hidden words.
4. Learn
Calculate Loss and update Millions of parameters via Backpropagation.
5. Repeat
Iterate millions of times on high-end GPUs.
The Training Loop

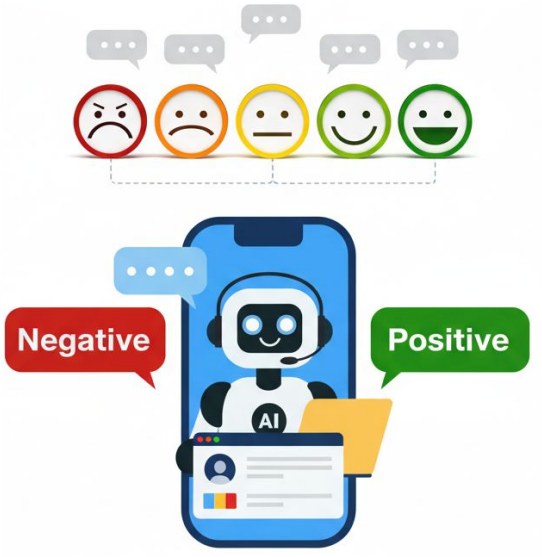
Real-World Application: The Google Search Revolution



The Problem: Before BERT, Google struggled with prepositions. Previous models missed the importance of "to".

The Solution: BERT understands that "to" dictates the direction of travel, providing correct visa info.

Real-World Application: Business Intelligence



Sentiment Analysis: Categorizing thousands of customer reviews into Positive/Negative instantly.

Text Classification: Routing support tickets to the right department or detecting spam/fraud.

https://abusix.com/blog/the-evolution-of-spam-the-present-part-2-of-3/

Real-World Application: Information Extraction & QA

Question Answering: Extracting precise answers from long documents (e.g., Virtual Assistants).

Hand on (Day 1: Afternoon)

Document Processing & Embeddings

- **Document Intelligence:** Learn to ingest and process various file formats (PDF, TXT) within the LangChain ecosystem.
- **Optimization Strategies:** Master Text Splitting (Chunking) techniques to effectively maintain context for AI.
- **Vector Embeddings:** Understand how to convert human language into high-dimensional vectors for semantic representation.
- **Semantic Search:** Build a retrieval system that understands meaning rather than just keyword matching.
- **Evaluation & Tuning:** Analyze the impact of "k" (number of retrieved chunks) on search accuracy and learn to balance performance with token costs and relevance.

Thank You